

Московский Авиационный Институт  
(Национальный Исследовательский Университет)  
Институт №8 “Компьютерные науки и прикладная математика”  
Кафедра №806 “Вычислительная математика и программирование”

**Лабораторная работа №1 по курсу**  
**«Операционные системы»**

Группа: М8О-207БВ-25

Студент: Михеева А.В.

Преподаватель: Бахарев В.Д.

Оценка: \_\_\_\_\_

Дата: 26.09.26

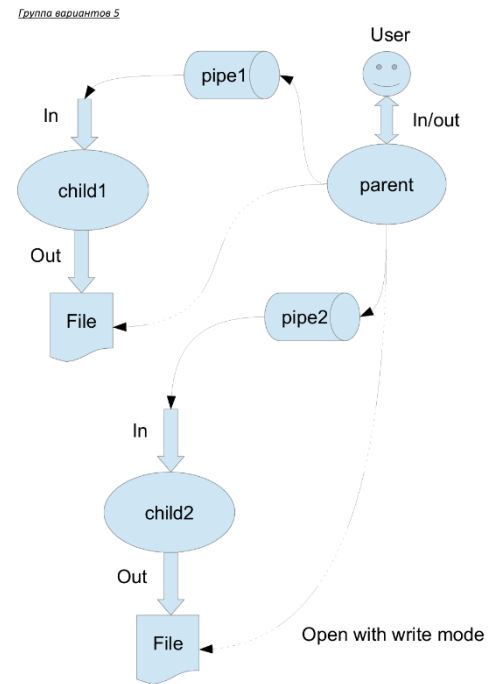
Москва, 2026

# Постановка задачи

## Вариант 20.

Родительский процесс создает два дочерних процесса. Первой строкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия File с таким именем на запись для child1. Аналогично для второй строки и процесса child2. Родительский и дочерний процесс должны быть представлены разными программами. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1 или в pipe2 в зависимости от правила фильтрации. Процесс child1 и child2 производят работу над строками. Процессы пишут результаты своей работы в стандартный вывод.

Правило фильтрации: строки длины больше 10 символов отправляются в pipe2, иначе в pipe1. Дочерние процессы инвертируют строки.



## Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; — создает дочерний процесс. Родительский процесс получает PID ребенка, ребенок получает 0.
- `int pipe(int pipefd[2])`; — создает неименованный канал для передачи данных между процессами. Индекс 0 отвечает за конец для чтения, 1 — конец для записи.
- `int dup2(int oldfd, int newfd)`; — переназначает файловый дескриптор. Копирует `oldfd` в `newfd`, закрывая `newfd`, если он был открыт.
- `int execv(const char *pathname, char *const argv[])`; — заменяет образ текущего процесса новой программой.
- `int open(const char *pathname, int flags, mode_t mode)`; — открывает или создает файл.
- `ssize_t read(int fd, void *buf, size_t count)`; — читает данные из файлового дескриптора.
- `ssize_t write(int fd, const void *buf, size_t count)`; — записывает данные в файловый дескриптор.
- `int close(int fd)`; — закрывает файловый дескриптор.
- `pid_t waitpid(pid_t pid, int *status, int options)`; — ожидает завершения дочернего процесса.
- `ssize_t readlink(const char *pathname, char *buf, size_t bufsiz)`; — читает содержимое символической ссылки.
- `void exit(int status)`; — завершает выполнение процесса с указанным статусом.

Программа состоит из двух отдельных исполняемых файлов: `parent` и `children`. Родительский процесс (`parent`) запускается пользователем в терминале.

Сначала `parent` определяет директорию, в которой находится его исполняемый файл, с помощью `readlink("/proc/self/exe", ...)` и обрезки имени файла до последнего слэша, тем самым

позволяя построить полный путь к дочерней программе children, которая лежит в той же директории. Путь собирается вручную посимвольным копированием в буфер path.

Затем создаётся два канала (pipe1 и pipe2) с помощью системного вызова pipe. Каждый канал представляет собой массив из двух дескрипторов: 0 отвечает за чтение, 1 — запись. Из стандартного ввода читаются две строки (имена файлов для child1 и child2 соответственно), введенные пользователем. Символ перевода строки в конце каждого имени заменяется на нулевой символ. Родитель открывает оба файла на запись с помощью open, получая дескрипторы fd1 и fd2.

Далее parent дважды вызывает fork. В дочерних процессах выполняются аналогичные действия. Закрываются ненужные концы каналов: запись этого процесса и оба канала другого процесса, а также дескриптор файла другого процесса. Затем через dup читающий конец pipe подменяет стандартный ввод, а дескриптор fd подменяет стандартный вывод. После этого оставшиеся дескрипторы (pipe с индексом 0 и fd) закрываются. Затем вызывается execv для запуска программы children без дополнительных аргументов. Если execv вернул управление — произошла ошибка, и процесс завершается с сообщением об ошибке. После двух fork читающие концы обоих каналов (pipe1[0] и pipe2[0]) и дескрипторы файлов fd1 и fd2 закрываются.

Затем parent входит в цикл чтения строк из стандартного ввода. Определяется длина строки без учета завершающего символа '\n'. Если длина больше 10 символов, строка записывается в pipe2 (для child2), иначе — в pipe1 (для child1). Цикл завершается, когда пользователь вводит пустую строку или нажимает Ctrl+D.

После выхода из цикла parent закрывает пишущие концы обоих каналов. Это приводит к тому, что в дочерних процессах read из стандартного ввода возвращает 0 (конец файла), и цикл чтения в них завершается. Дочерние процессы выполняют close для файла и возвращают 0. parent вызывает wait(NULL) дважды, ожидая завершения обоих дочерних процессов, и завершается сам.

После запуска через execv стандартный вывод children уже перенаправлен на файл через dup2, выполненный parent. Программа читает строки из стандартного ввода (который перенаправлен на соответствующий канал), для каждой строки убирает завершающий символ '\n' из подсчета длины, инвертирует строку, и записывает результат в стандартный вывод, который фактически идет в файл. После записи строки добавляется символ '\n'. Когда read возвращает 0 (родитель закрыл канал), цикл while завершается, и программа возвращает 0.

## Код программы

```
twentyeighth@connor:~/lab$ cat parent.c
#include <stdint.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <fcntl.h>
#include <string.h>

static char CHILD_PROGRAM_NAME[] = "children";

int main(int argc, char** argv) {
    (void)argc;
    (void)argv;

    char prospath[1024];
    ssize_t len = readlink("/proc/self/exe", prospath, sizeof(prospath) - 1);
    if (len == -1) {
```

```

        const char msg[] = "error: failed to read full program path\n";
        write(STDERR_FILENO, msg, sizeof(msg));
        exit(EXIT_FAILURE);
    }
    while (progp[proglen] != '/') proglen--;
    progp[proglen] = '\0';

    int pipe1[2], pipe2[2];
    if (pipe(pipe1) == -1 || pipe(pipe2) == -1) {
        const char msg[] = "error: failed to create pipe\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    char path[1024];

    size_t i;
    for (i = 0; i < strlen(progp) && i + 1 < sizeof(path); i++)
        path[i] = progp[i];
    if (i + 1 < sizeof(path)) path[i++] = '/';
    for (size_t j = 0; j < strlen(CHILD_PROGRAM_NAME) && i + 1 < sizeof(path); j++)
        path[i++] = CHILD_PROGRAM_NAME[j];
    path[i] = '\0';

    char file1[256];
    char file2[256];

    ssize_t x = read(STDIN_FILENO, file1, sizeof(file1) - 1);
    if (x <= 0) {
        const char msg[] = "error: failed to read file1 name\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    file1[x] = '\0';
    if (x > 0 && file1[x - 1] == '\n') file1[x - 1] = '\0';

    x = read(STDIN_FILENO, file2, sizeof(file2) - 1);
    if (x <= 0) {
        const char msg[] = "error: failed to read file2 name\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
    file2[x] = '\0';
    if (x > 0 && file2[x - 1] == '\n') file2[x - 1] = '\0';

    int fd1 = open(file1, O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (fd1 == -1) {
        const char msg[] = "error: failed to open file1\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    int fd2 = open(file2, O_WRONLY | O_CREAT | O_TRUNC, 0600);
    if (fd2 == -1) {
        const char msg[] = "error: failed to open file2\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    const pid_t child1 = fork();

    switch (child1) {
    case -1: {
        const char msg[] = "error: failed to spawn new process [child1]\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
    }
    }

```

```

        exit(EXIT_FAILURE);
    } break;

    case 0: {

        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
        close(fd2);

        dup2(pipe1[0], STDIN_FILENO);
        close(pipe1[0]);

        dup2(fd1, STDOUT_FILENO);
        close(fd1);

        char* const args[] = { CHILD_PROGRAM_NAME, NULL };

        int32_t status = execv(path, args);
        if (status == -1) {
            const char msg[] = "error: failed to exec into new executable image
[child1]\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            exit(EXIT_FAILURE);
        }
    } break;
    default:
        break;
}

const pid_t child2 = fork();

switch (child2) {
case -1: {
    const char msg[] = "error: failed to spawn new process [child2]\n";
    write(STDERR_FILENO, msg, sizeof(msg) - 1);
    exit(EXIT_FAILURE);
} break;

case 0: {

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[1]);
    close(fd1);

    dup2(pipe2[0], STDIN_FILENO);
    close(pipe2[0]);

    dup2(fd2, STDOUT_FILENO);
    close(fd2);

    char* const args[] = { CHILD_PROGRAM_NAME, NULL };

    int32_t status = execv(path, args);
    if (status == -1) {
        const char msg[] = "error: failed to exec into new executable image
[child2]\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }
} break;
default:
    break;
}

```

```

char buf[4096];
ssize_t bytes;

close(pipe1[0]);
close(pipe2[0]);
close(fd1);
close(fd2);

while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
    if (buf[0] == '\n') break;

    size_t linelen = (size_t)bytes;
    if (linelen > 0 && buf[linelen - 1] == '\n') linelen--;

    int t = (linelen > 10) ? pipe2[1] : pipe1[1];
    write(t, buf, bytes);
}

close(pipe1[1]);
close(pipe2[1]);

wait(NULL);
wait(NULL);

return 0;
}
twentyeighth@connor:~/lab$ cat children.c
#include <stdint.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char** argv) {
    char buf[4096];
    ssize_t bytes;

    (void)argc;
    (void)argv;

    while ((bytes = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        ssize_t len = bytes;
        if (len > 0 && buf[len - 1] == '\n') len--;

        for (ssize_t i = 0; i < len / 2; i++) {
            char t = buf[i];
            buf[i] = buf[len - 1 - i];
            buf[len - 1 - i] = t;
        }

        write(STDOUT_FILENO, buf, len);
        write(STDOUT_FILENO, "\n", 1);
    }

    if (bytes < 0) {
        const char msg[] = "error: failed to read from stdin\n";
        write(STDERR_FILENO, msg, sizeof(msg) - 1);
        exit(EXIT_FAILURE);
    }

    return 0;
}

```

## Протокол работы программы

```
twentyeighth@connor:~/lab$ ls
```

```

children children.c parent parent.c
twentyeighth@connor:~/lab$ ./parent
file1.txt
file2.txt
helloworld
Zatanna
Amaze amaze amaze
Proper nouns are a headache. If you're learning German from a guy named Hans, you just
call him Hans. But I literally can't make the noises Rocky makes and vice versa. So
when one of us tells the other about a name, the other one has to pick or invent a word
to represent that name in their own language. Rocky's actual name is a sequence of
notes - he told it to me once but it has no meaning in his language, so I stuck with
"Rocky." But my name is actually an English word. So Rocky just calls me the Eridian
word for "grace." Anyway, now I have to come up with an English word that means
"Rocky's spouse." "Adrian," I say. Why not? "Human word is 'Adrian.'"

twentyeighth@connor:~/lab$ cat file1.txt
dlrowolleh
annataZ
twentyeighth@connor:~/lab$ cat file2.txt
ezama ezama ezamA
"'nairdA' si drow namuH" ?ton yHW .yas I ",nairdA" ".esuops s'ykcoR" snaem taht drow
hsilgnE na htiw pu emoc ot evah I won ,yawynA ".ecarg" rof drow naidirE eht em sllac
tsuj ykcoR oS .drow hsilgnE na yllautca si eman ym tuB ".ykcoR" htiw kcuts I os
,egaugnal sih ni gninaem on sah ti tub ecno em ot ti dlot eh - seton fo ecneuqes a si
eman lautca s'ykcoR .egaugnal nwo rieht ni eman taht tneserper ot drow a tnevni ro kcip
ot sah eno rehto eht ,eman a tuoba rehto eht silet su fo eno nehW oS .asrev eciv dna
sekam ykcoR sesion eht ekam t'nac yllaretil I tuB .snaH mih llac tsuj uoy ,snaH deman
yug a morf namreG gninrael er'uoy fI .ehcadaeh a era snuon reporP

```

## Вывод

В ходе лабораторной работы я составила и отладила программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними в Linux. Запрет на часть стандартных библиотек Си не вызвало столько проблем, сколько казалось изначально. Основная сложность возникла при отладке завершения процессов: без закрытия лишних концов каналов в родителе программа зависала в wait. В целом лабораторная дала хорошее понимание управления процессами в ОС.